

ADVANCED DEEP LEARNING COMPANION READER

Variational Autoencoders

From basic autoencoders to VAEs and VQ-VAEs

Session 13 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation. Every slide example is worked out in full, step by step. Colour-coded boxes guide you. The blue “intuition” box states the core idea in one breath. The amber “everyday picture” box ties it to real life before any math. The green “worked example” box solves a problem with real numbers and no skipped steps. The red “watch out” box flags the common mistake. The purple “key takeaway” box is the one sentence to remember. Read it alongside the slides, in order.

1 Autoencoder (Recap)

Autoencoders compress data into a smaller representation and then try to reconstruct it. The network has an encoder that shrinks the input into a **latent space**, and a decoder that blows it back up. But standard autoencoders just want to copy the input. They do not care how the latent space is organized, leading to an irregular, patchy map where points between examples decode into noise.

The intuition

An autoencoder is like packing a suitcase in a rush. You cram everything in to make it fit (encoding) and pull it out when you arrive (decoding). But the space inside the suitcase is a mess. If you reach into a random spot, you might pull out a shoe mixed with a toothbrush.

★ Everyday picture

Imagine taking a photo, shrinking it to a tiny thumbnail, and then asking an artist to paint the full picture back from just the thumbnail. The thumbnail is the latent representation. It forces the system to keep only the most vital details.

Standard autoencoders lose “semantics” in the gaps. We want to be able to pick *any* point in the latent space and decode it into something that looks real. A standard autoencoder gives us an **irregular latent space**.

Worked example: A standard autoencoder gap

If we encode an image of a cat to $z_1 = [0, 1]$ and a dog to $z_2 = [1, 0]$.

Step 1. Pick a point in between. Let’s pick $z = [0.5, 0.5]$.

Step 2. Decode it. Because the autoencoder only trained on the exact points for the cat and dog, the decoder does not know what to do with $[0.5, 0.5]$.

Step 3. Result. The output is usually a blurry mess, not a half-cat half-dog.

✗ **Watch out**

Standard autoencoders are great for compression and denoising, but terrible for generation. You cannot just sample random numbers and expect a realistic output because the space is full of holes.

✓ **Key takeaway**

A standard autoencoder learns to copy inputs through a bottleneck. But its latent space is irregular and full of gaps. This makes it useless for generating new data.

ML/AI connection. Basic autoencoders are used for denoising images or anomaly detection. However, for generation, we need to enforce structure. This leads us to Variational Autoencoders.

Standard Autoencoder



Variational Autoencoder

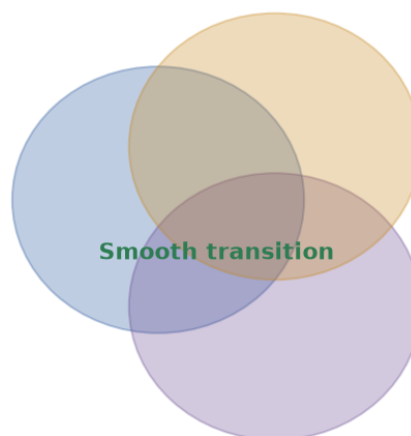


Figure 1: Standard autoencoder vs VAE latent space. The standard AE has gaps between clusters, while the VAE forces a continuous, smooth region.

2 Entropy and KL-Divergence

To fix the autoencoder, we need a way to measure and control the shape of our latent space. We use two ideas from information theory: **Entropy** and **KL-Divergence**.

The intuition

Entropy measures how messy or unpredictable a system is. KL-Divergence measures how different two distributions are. We use KL-Divergence as a penalty to force our messy latent space to look like a clean, simple bell curve.

★ Everyday picture

Think of entropy as the clutter in your room. A perfectly organized room has zero entropy. KL-Divergence is like comparing your room's layout to a picture in an IKEA catalog. The bigger the difference, the higher the penalty.

Entropy is the expected value of information for an event. It measures disorder. For a pure, perfectly predictable system, entropy is zero.

KL-Divergence (Kullback-Leibler Divergence) measures the difference in information content between two distributions, p and q .

$$D_{KL}(p \parallel q) = \mathbb{E}_{p(x)} \left[\log \frac{p(x)}{q(x)} \right] \quad (1)$$

Here, the expectation $\mathbb{E}$ is computed with respect to $p(x)$. KL-Divergence is always positive, and it is *not* symmetric: $D_{KL}(p \parallel q)$ is not the same as $D_{KL}(q \parallel p)$.

Worked example: KL-Divergence of two simple probabilities

Let p and q be two distributions over two states, A and B . Let $p(A) = 0.5, p(B) = 0.5$, and let $q(A) = 0.8, q(B) = 0.2$.

Step 1. Write the formula. $D_{KL}(p \parallel q) = p(A) \log \frac{p(A)}{q(A)} + p(B) \log \frac{p(B)}{q(B)}$.

Step 2. Plug in the numbers. $D_{KL} = 0.5 \log \frac{0.5}{0.8} + 0.5 \log \frac{0.5}{0.2}$.

Step 3. Calculate. $\log(0.625) \approx -0.470$ and $\log(2.5) \approx 0.916$.

Step 4. Combine. $0.5(-0.470) + 0.5(0.916) = -0.235 + 0.458 = 0.223$.

Step 5. Result. The divergence is 0.223, which is positive, as expected.

✗ Watch out

Never treat KL-Divergence as a true “distance” metric, because it is not symmetric. The order matters!

✓ Key takeaway

Entropy measures unpredictability. KL-Divergence measures how far one probability distribution diverges from another, providing a way to penalize models that stray from a target shape.

ML/AI connection. KL-Divergence is the core of the Variational Autoencoder loss function. It acts as a regularizer, forcing the network's latent groupings to match a standard Gaussian distribution, ensuring no gaps.

3 Variational Autoencoders

A **Variational Autoencoder (VAE)** solves the gap problem by changing what the encoder outputs. Instead of outputting a single point, the encoder outputs parameters of a distribution. Usually this is the mean μ and variance σ^2 of a Gaussian curve.

The intuition

Instead of placing the image exactly at one spot on the map, the VAE places it in a blurry cloud. This forces the clouds for different images to overlap slightly, covering the gaps and making the entire space smooth.

★ Everyday picture

A VAE draws a circle and says the arrow will land inside it. The circles overlap. There are no empty spots on the target.

During training, we sample a random point z from this distribution and pass it to the decoder. Because the decoder must learn to reconstruct the original image from *any* point in that cloud, it learns a smooth, continuous mapping.

Worked example: Sampling from the latent cloud

Suppose the encoder looks at a cat image and outputs a mean $\mu = [0, 0]$ and variance $\sigma^2 = [1, 1]$.

Step 1. Define the distribution. $q(z|x)$ is a Gaussian with mean $[0, 0]$ and variance $[1, 1]$.

Step 2. Sample a random point. We draw a random z from this distribution. Let's say we get $z = [0.2, -0.5]$.

Step 3. Decode. We pass $z = [0.2, -0.5]$ to the decoder, which reconstructs the cat.

Step 4. Repeat. Next time we see the same cat, we might sample $z = [-0.1, 0.4]$. The decoder must reconstruct the cat from this point too.

✗ Watch out

You cannot backpropagate through a random sampling step. To fix this, VAEs use the **reparameterization trick**: $z = \mu + \sigma \odot \epsilon$, where ϵ is random noise. The randomness is pushed to the side, allowing gradients to flow through μ and σ .

✓ Key takeaway

A VAE maps inputs to probability distributions rather than fixed points. This forces the latent space to be continuous. It allows us to generate new data by sampling.

ML/AI connection. VAEs are a foundational generative model. They are used to generate new faces and create interpolations (morphing one image into another). They also serve as the basis for more advanced models like Diffusion models.

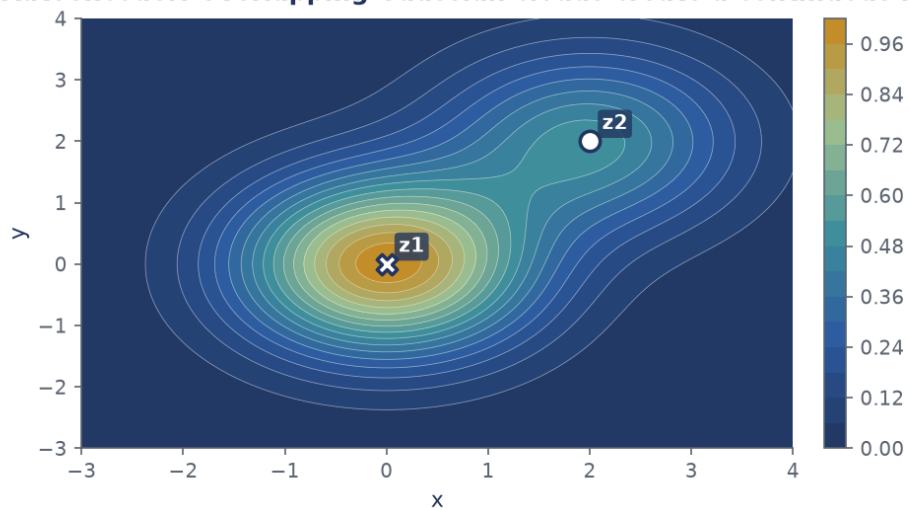
Variational Autoencoder: Overlapping Gaussian clouds create a continuous manifold.

Figure 2: The VAE latent space. By forcing the latent representations into overlapping Gaussian clouds, the space becomes smooth and regular.

4 Variational Lower Bound (ELBO)

To train a VAE, we want to maximize the likelihood of our data, $p_\theta(x)$. But computing this directly is impossible (intractable) because we would have to check every possible latent variable z .

The intuition

We cannot measure the exact height of the mountain (the true data likelihood). But we can measure the height of a base camp that is always below the peak. If we push the base camp higher, we guarantee the peak is at least that high.

★ Everyday picture

Imagine trying to count every grain of sand on a beach to find its weight. It is impossible. Instead, you weigh a few buckets and use math to find a guaranteed minimum weight. You maximize this lower bound instead of counting every grain.

Instead of maximizing $p_\theta(x)$, we maximize a tractable stand-in called the **Evidence Lower Bound (ELBO)**. The ELBO has two parts:

$$\text{ELBO} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p(z)) \quad (2)$$

Let's break this down: 1. **Reconstruction loss:** $\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)]$. This rewards the model for accurately reconstructing the input x from the latent sample z . 2. **KL Divergence penalty:** $D_{KL}(q_\phi(z|x) \parallel p(z))$. This penalizes the encoder if its learned distribution $q_\phi(z|x)$ strays too far from the prior $p(z)$ (usually a standard normal distribution with mean 0 and variance 1).

Worked example: Calculating the ELBO

Assume the reconstruction loss is 10 (higher is better likelihood), and the KL penalty is 2.

Step 1. Identify the terms. Reconstruction = 10, KL Penalty = 2.

Step 2. Apply the formula. ELBO = Reconstruction – KL Penalty.

Step 3. Calculate. ELBO = 10 – 2 = 8.

Step 4. Result. The lower bound on our log-likelihood is 8. During training, gradient descent adjusts the weights to push this number as high as possible.

✗ Watch out

Do not think of the KL term as just an error to minimize. It is a *regularizer*. Without it, the VAE would shrink the variance to zero and become a standard autoencoder, destroying the ability to generate new data.

✓ Key takeaway

Because the true data likelihood is impossible to compute, VAEs are trained by maximizing the ELBO. This balances good reconstruction against keeping the latent space organized.

ML/AI connection. The ELBO is the workhorse objective for variational inference across

machine learning, appearing not just in VAEs but in Bayesian neural networks and diffusion models.

5 Beta-VAE

The standard VAE is great, but its latent dimensions might mix features together. For example, changing z_1 might change both a person's smile and their head pose at the same time. We want **disentangled representations**, where each dimension of z controls one interpretable feature (like *only* head pose, or *only* lighting).

The intuition

Disentanglement is like having separate volume, bass, and treble knobs on a stereo. If the knobs were tangled, turning up the volume might also randomly change the bass. Beta-VAE forces the network to learn separate, independent knobs for the features.

★ Everyday picture

Imagine packing a suitcase (again), but this time you are penalized heavily if you put shoes and toothbrushes in the same compartment. You are forced to organize the suitcase into clear, separate sections for each type of item.

To force disentanglement, **Beta-VAE** (β -VAE) simply adds a multiplier, β , to the KL-Divergence penalty in the ELBO:

$$L = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \beta D_{KL}(q_\phi(z|x) \parallel p(z)) \quad (3)$$

When $\beta = 1$, this is just a standard VAE. When $\beta > 1$, we put stronger pressure on the latent distribution to match the independent, un-correlated prior. This heavy penalty forces the network to store information as efficiently as possible, aligning features with the independent axes of the latent space.

Worked example: Adjusting the Beta penalty

Suppose the standard KL penalty is 2.0.

Step 1. Standard VAE. $\beta = 1$. The penalty is $1 \times 2.0 = 2.0$.

Step 2. Beta-VAE. We set $\beta = 5$. The new penalty is $5 \times 2.0 = 10.0$.

Step 3. Result. The network will try much harder to reduce the KL divergence to satisfy the larger penalty, forcing the latent variables to become independent.

✗ Watch out

If you make β too high, the network ignores the reconstruction loss completely. It will just output a perfect, independent Gaussian that decodes into blurry, meaningless mush.

✓ Key takeaway

Beta-VAE adds a weight $\beta > 1$ to the KL penalty, forcing the network to learn disentangled, interpretable features where each axis controls one specific trait.

ML/AI connection. Disentangled representations are highly sought after for controllable generation and fairness. If a model disentangles “skin color” from “criminality prediction” or “lighting” from “identity,” it is much safer and more useful to deploy.

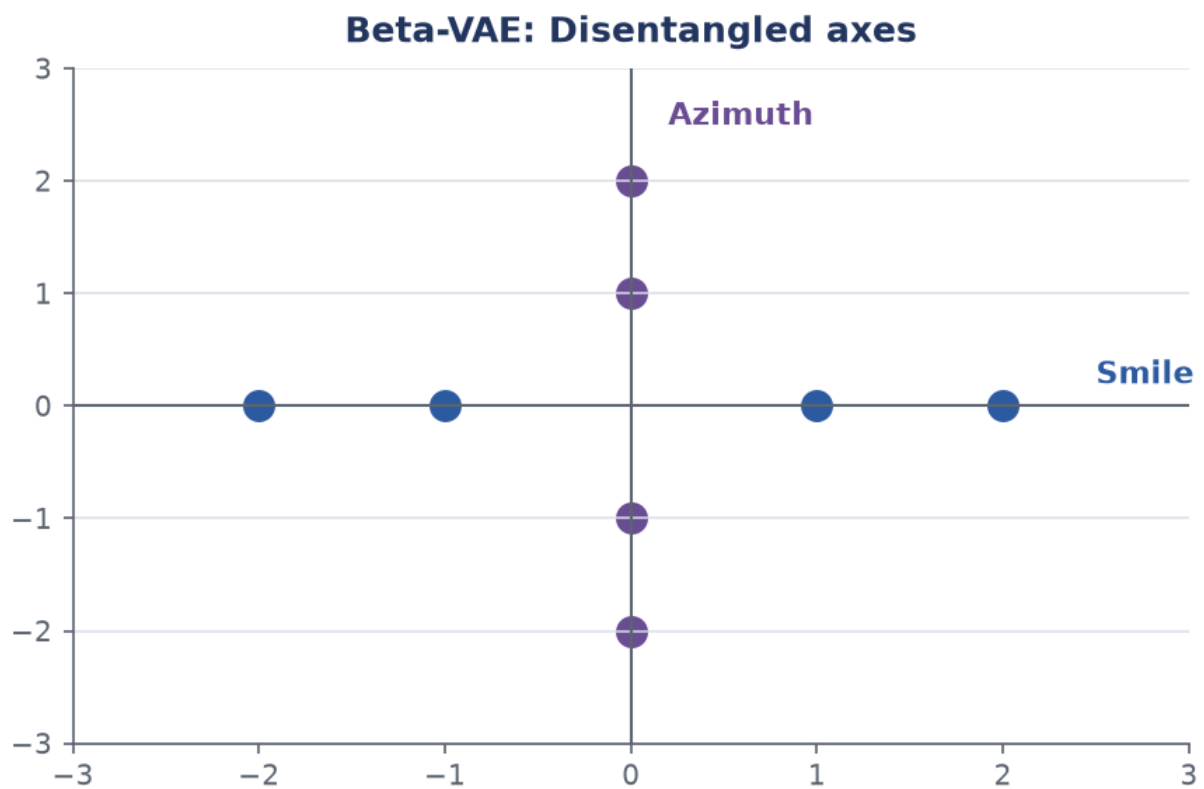


Figure 3: Disentangled latent traversal. Moving along one axis changes only one feature, like azimuth (rotation), while keeping everything else fixed.

6 VQ-VAE

Standard VAEs use continuous numbers for the latent space (like $z = 0.432$). But many things in the real world are discrete: words, phonemes in speech, or categories. The **Vector Quantized VAE (VQ-VAE)** replaces the continuous latent space with a discrete set of codes.

The intuition

Instead of saying a color is $RGB(12, 145, 23)$, you just say “Blue.” VQ-VAE takes the messy, continuous output of the encoder and snaps it to the nearest item in a predefined dictionary.

★ Everyday picture

Think of a paint-by-numbers kit. The real world has infinite colors, but the kit restricts you to exactly 32 specific paint pots. The VQ-VAE looks at a continuous color and rounds it to the nearest available paint pot.

The VQ-VAE has a learned dictionary (the embedding space) of vectors, e_k . The encoder outputs a continuous vector, $z_e(x)$. Then, we find the dictionary vector e_i that is closest to $z_e(x)$ and pass e_i to the decoder.

The gradient cannot flow through this “snap to nearest” step, because rounding is not differentiable. To fix this, VQ-VAE uses a **stop-gradient** operator, sg . During the backward pass, it copies the gradients straight from the decoder to the encoder, ignoring the quantization step.

$$L = \text{Reconstruction} + \|sg(z_e(x)) - e\|_2^2 + \beta \|z_e(x) - sg(e)\|_2^2 \quad (4)$$

The second term moves the dictionary vectors towards the encoder outputs. The third term (the commitment loss) makes sure the encoder commits to an embedding and does not grow infinitely large.

Worked example: Snapping to the nearest vector

Let the encoder output $z = [0.2, 0.9]$. We have a dictionary with two vectors: $e_1 = [0, 0]$ and $e_2 = [0, 1]$.

Step 1. Calculate distance to e_1 . Distance to $[0, 0]$ is $\sqrt{0.2^2 + 0.9^2} = \sqrt{0.04 + 0.81} = \sqrt{0.85} \approx 0.92$.

Step 2. Calculate distance to e_2 . Distance to $[0, 1]$ is $\sqrt{0.2^2 + (-0.1)^2} = \sqrt{0.04 + 0.01} = \sqrt{0.05} \approx 0.22$.

Step 3. Pick the closest. e_2 is much closer.

Step 4. Result. We replace the continuous z with the discrete dictionary vector e_2 , and pass e_2 to the decoder.

✗ Watch out

Because the latent space is discrete, you can no longer sample by just picking random numbers from a Gaussian. You must train a separate prior model (like a PixelCNN) over the discrete codes to generate new samples.

✓ Key takeaway

VQ-VAE uses a dictionary of embeddings and a stop-gradient trick. This allows it to learn discrete latent representations. It prevents posterior collapse and works perfectly for speech or text.

ML/AI connection. VQ-VAEs are massive in modern AI. They are the foundation for generating high-quality speech and transferring voice styles. They are also a key component in large text-to-image models to compress images into discrete tokens.

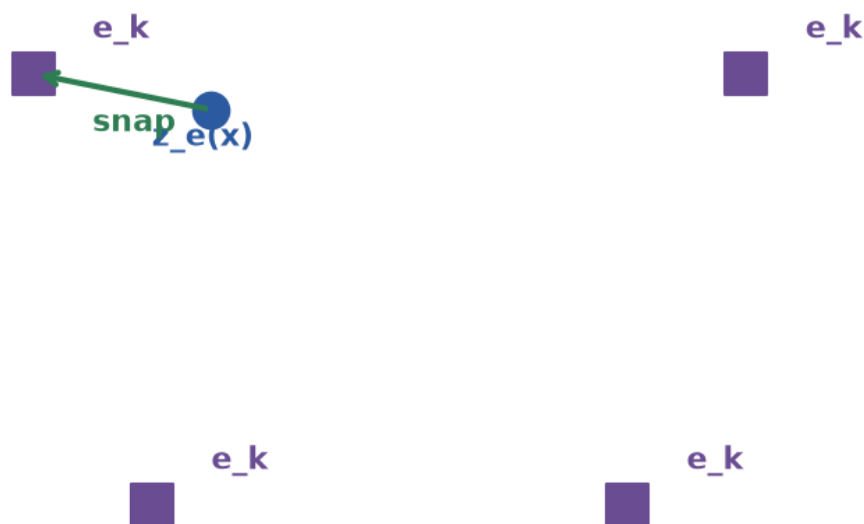
VQ-VAE: Snapping to the nearest discrete vector

Figure 4: The VQ-VAE bottleneck. The continuous encoder output $z_e(x)$ snaps to the nearest discrete vector e_2 in the dictionary.

7 VQ-VAE-2

The **VQ-VAE-2** scales up the original idea by using a hierarchy of latent maps. Instead of one bottleneck, it has multiple layers of discrete codes.

The intuition

If you are describing a face, you first describe the big picture: “It is a woman with dark hair.” Then you add the details: “She has a small mole on her left cheek.” The top layer handles the global shape; the bottom layer handles the fine details.

★ Everyday picture

Think of compressing a video. The first pass captures the rough blocks of color so you can see the scene. A second, higher-resolution pass fills in the sharp edges and textures. VQ-VAE-2 does exactly this, but using discrete codes at every level.

An image is compressed into a tiny map (e.g., 32×32) for global structure, and a larger map (e.g., 64×64) for local details. The decoder uses both maps to reconstruct the image. This hierarchical approach allows VQ-VAE-2 to generate incredibly high-resolution, realistic images (like the FFHQ face dataset) that rival GANs, but without the training instability.

Worked example: Hierarchical compression ratios

Let the original image be $256 \times 256 \times 3$ (color).

Step 1. Calculate original size. $256 \times 256 \times 3 = 196,608$ numbers.

Step 2. Bottom latent map. Size $64 \times 64 = 4,096$ discrete codes.

Step 3. Top latent map. Size $32 \times 32 = 1,024$ discrete codes.

Step 4. Total latents. $4,096 + 1,024 = 5,120$ codes.

Step 5. Compression ratio. $196,608 / 5,120 \approx 38.4$ times smaller than the original image, while preserving all the details.

Watch out

While VQ-VAE-2 avoids GAN mode collapse, it is much slower to sample from. Generating an image requires running the autoregressive prior (like PixelCNN) step-by-step over the discrete codes, which takes time.

✓ Key takeaway

VQ-VAE-2 uses a hierarchy of discrete latent maps to separate global structure from local details, enabling the generation of extremely high-quality, high-resolution images.

ML/AI connection. Hierarchical discrete autoencoders compress images for powerful autoregressive or diffusion models. They separate the heavy lifting of structure generation from painting fine pixels.

Glossary & symbols

Key terms.

latent space	the compressed representation inside an autoencoder.
entropy	a measure of how unpredictable or messy a distribution is.
KL-Divergence	a measure of how different one probability distribution is from another.
reparameterization trick	a math trick to allow backpropagation through random sampling.
ELBO	Evidence Lower Bound; the objective function we maximize to train a VAE.
disentangled representation	a latent space where each axis controls exactly one interpretable feature.
stop-gradient	an operator that acts as the identity forward, but stops gradients from flowing backward.

Symbols at a glance.

Symbol	Meaning
$D_{KL}(p \parallel q)$	KL-Divergence of q from p
μ	the mean of a Gaussian distribution
σ^2	the variance of a Gaussian distribution
$p(z)$	the prior distribution of the latent space (usually standard normal)
$q_\phi(z x)$	the encoder's approximate posterior distribution
$p_\theta(x z)$	the decoder's likelihood distribution
β	the weight on the KL penalty in Beta-VAE
$z_e(x)$	the continuous output of the encoder before quantization
e_k	a discrete vector in the VQ-VAE dictionary
$sg()$	the stop-gradient operator